ANONYMIZED PRODUCT DESIGN EXPLORATION

Research & Product Brief

How a security team might trace and govern risky AI models across a software supply chain.

Context

AI models may be embedded inside packages or container images. A single detected model can therefore appear across several repositories, artifacts, projects and environments. Actions affect the containing software artifacts, so the operator needs to understand reach before deciding what to govern.

Research objective

Understand the technical structure around a detected model, identify the information required for a safe decision, and define how the interaction should change as the potential impact grows.

Research approach

01	02	03
Domain review	Workflow mapping	Scenario analysis
Mapped the relationship between projects, repositories, artifacts and embedded models.	Followed the investigation from detection through context gathering and governance.	Compared a localized finding with a model appearing across multiple environments.

Research boundary

This was a concept exploration. The findings below are design hypotheses derived from domain research and scenario mapping. They were not validated through customer interviews or production usage.

SYNTHESIS

The investigation reverses the system hierarchy

The software is organized from organization context down to the model. The investigation begins with the model and moves outward to every place that contains it.

SYSTEM STRUCTURE	INVESTIGATION PATH
Project > Repository > Artifact > Model This hierarchy explains ownership and where an action can be applied.	Model > Artifacts > Repositories > Projects This path explains reach and the potential blast radius of a decision.

What the research suggested

SCOPE	STATE	RISK
A model is not the action target Governance applies to the software artifacts that contain the model.	Parents need an aggregate status Conflicting child states should appear as Mixed at the parent level.	One workflow cannot fit every scope Broader actions need more context, impact analysis and confirmation.

Design principles

Scope awareness. Show what will be affected before the action is taken.
Progressive risk. Increase context and safeguards as reach grows.
Governance in context. Keep the organizational hierarchy visible throughout the decision.

Detect > Understand > Govern

PRODUCT DEFINITION

Governance should scale with scope and risk

The proposed workspace keeps organizational reach, environment, governance state and technical context visible in one investigation.

Core requirements

INVESTIGATE	UNDERSTAND	GOVERN
Trace every occurrence Expand projects and repositories to see each containing artifact and its environment.	**Keep model context available** Use a collapsible details panel for security, origin, ownership, usage and technical metadata.	**Match friction to reach** Use lightweight confirmation for one artifact and impact analysis for broad actions.

System rules

AGGREGATION	PROPAGATION
Conflicting child states become Mixed A parent communicates that its descendants do not share one governance state.	**Higher-level actions cascade** A project-level action affects the repositories and containing artifacts below it.

Questions to validate

1. Does the proposed hierarchy match how security teams investigate model risk?
2. Which metadata is required before an operator feels confident taking action?
3. When should a broad action require approval from another role?
4. What audit history and rollback controls are required after governance changes?

Possible success signals

Time to understand total reach, errors in selecting the intended scope, abandoned governance actions, time from detection to resolution, and operator confidence before confirmation.